

Assignment

CMPE452 Advanced Deep Learning Techniques and Applications (2025-2026 Spring)

Part A (PoG: 10%)

Overview

In this part, students are expected to develop, train, and evaluate a CNN-based classification model using PyTorch on Google Colab. Students will select their own dataset from Kaggle. The selected dataset must be suitable for a classification problem. Regression problems will not be accepted for this assignment.

Students should compare model performance by experimenting with different CNN architectures and hyperparameters on their selected dataset. For example, they are expected to try different options such as the number of layers, number of filters, kernel size, pooling strategy, dropout usage, optimizer type, batch size, learning rate, and activation functions.

All development and experimentation must be conducted on Google Colab.

Dataset Details

Students are required to select their own dataset from Kaggle for this assignment.

The selected dataset must include a classification problem. The dataset should be suitable for use with a CNN model. Image classification datasets are especially appropriate for this assignment.

The selected dataset must be clearly introduced in the report. The following information should be provided:

- Name of the dataset
- Kaggle link of the dataset
- Number of classes in the dataset
- What the classes represent
- Number of samples in the dataset
- General structure of the data
- Format of the input data
- Target variable / class labels
- How the training, validation, and test split was performed

The selected dataset should contain a sufficient number of samples for model training and evaluation.

Assignment Instructions

1. Environment Setup on Google Colab

- Open a new notebook on Google Colab.
- Select GPU runtime to accelerate model training.
To do this, follow these steps:
Runtime > Change runtime type > GPU
- Install or import the necessary libraries in the notebook.
- PyTorch must be used during the model development process.
- Your code should be clear, organized, and well-documented.

2. Data Loading and Preprocessing

- Load your selected Kaggle dataset into the Google Colab environment.
- Convert the dataset into a format suitable for use with PyTorch.
- Split the dataset into training, validation, and test sets.
- Clearly show the training, validation, and test split in the notebook.
- Apply the necessary transformations for image data.

Possible preprocessing steps include:

- Resizing images
- Normalizing images
- Converting labels into the appropriate format
- Using data augmentation methods
- Creating training, validation, and test data loaders

Possible data augmentation methods include:

- Random rotation
- Random horizontal flip
- Random crop
- Color jitter
- Resize
- Normalize

The test data must only be used during the final evaluation stage.

Training on the test data is strictly prohibited and will be considered plagiarism.

3. Model Architecture

Students must develop a CNN-based deep learning model suitable for their selected classification dataset.

In this assignment, every student must use a Convolutional Neural Network (CNN) structure. Therefore, the developed model must include convolutional layer structures.

Students should first create a baseline CNN model and then try to improve model performance by experimenting with different architectures and hyperparameters.

An example CNN model may include the following components:

- Convolutional layer
- Activation function
- Pooling layer
- Dropout layer
- Fully connected layer
- Output layer

Students must conduct experiments by modifying the following hyperparameters:

- Convolutional kernel size
- Convolutional dropout
- Activation function
- Batch size
- Optimizer type
- Number of convolutional layers
- Fully connected hidden units
- Pooling kernel size
- Pooling stride
- Fully connected dropout
- Number of epochs

Detailed instructions on how to conduct the hyperparameter experiments and how to report the results have been shared on Moodle. Students must follow the document titled **“CMPE452 Hyperparameter Experiment Guidelines”** while conducting their experiments.

Students should explain and comparatively present the effects of their architectural and hyperparameter changes on model performance in the report.

4. Training the Model

Loss Function and Optimizer

Use CrossEntropyLoss as your loss function and experiment with optimizers such as SGD and Adam.

Training Procedure

- Implement a training loop that feeds batches of training data into the model, computes the loss, and updates the model weights.
- Validate the model on the validation set during training to monitor performance and adjust hyperparameters accordingly.
- Record and visualize metrics such as training loss, validation loss, and accuracy over epochs.

Experimentation

- Document the impact of various hyperparameters such as kernel size, number of layers, batch size, optimizer choice, and activation functions by plotting training and validation curves.
- Include comparisons showing how different settings such as pooling sizes, filter sizes, and dropout configurations affect model performance.

5. Evaluation

- Once training is complete, evaluate your final model on the test dataset.
- Compute key metrics including accuracy, precision, recall, and a confusion matrix.
- Discuss your experimental findings, including any challenges related to overfitting, convergence issues, or other observations from your hyperparameter experiments.

6. Reporting

Your final report should include the following sections:

Data Preparation

- Details on how the dataset was loaded, preprocessed, and split into training, validation, and test sets.

Model Architecture and Hyperparameter Choices

- A clear description of your baseline model architecture and any modifications you experimented with.
- Justification for your choices, such as why a specific pooling size or dropout configuration was selected.

Training and Experimentation

- Training curves for both loss and accuracy on the training and validation sets.
- Comparative analyses of different settings such as kernel sizes, optimizers, and activation functions with corresponding figures.

Final Evaluation

- Performance metrics on the test dataset with discussion.

Conclusion

- A summary of the key lessons learned, limitations encountered, and potential future improvements.

7. Submission Guidelines

- Submit the Google Colab notebook containing your full code and visualizations.
- Include your final report as a PDF summarizing your approach, experiments, and results, including visualizations.
- Ensure that your code is well-documented and follows PyTorch best practices.
- If you miss the deadline, you will receive zero.
- **You may benefit from AI for coding part, but you must properly modify the output yourself and explain where and how you used it. Otherwise, you will receive zero.**
- **AI must not be used in any part of the report writing process. The report must be written entirely by the student. If any AI usage is detected, you will receive zero.**
- DO NOT train on the test data. Training on the test data is considered plagiarism.
- The notebook and the report should be submitted individually via Moodle.
- **Deadline for Part A: 16.05.2025, 23:59.**

Part B

PoG: 10%

Problem: Interface Development and Presentation for the Trained Model

Overview

In this part, students are expected to develop an interface that can use the **CNN-based classification model** they developed in **Part A** and obtained the best result with.

The developed interface should allow the user to provide an appropriate input for the model, use the trained model to generate a prediction, and clearly display the model output to the user.

The aim of Part B is not only to train a model, but also to create a simple and functional system where the model developed in Part A can be used in practice.

Interface Requirements

The developed interface must have the following features:

- It should be able to receive an appropriate input from the user.
- It should be able to use the best model obtained in Part A.
- It should process the user input in a format suitable for the model.
- It should display the model prediction or output to the user.
- The interface output should be clear, understandable, and interpretable.

For example, a student working on an image classification problem should allow the user to upload an image through the interface and show which class the model predicts for that image.

Technologies That Can Be Used

Any programming language, framework, or technology can be used for interface development. The choice of technology is left to the student. The important point is that the developed interface must be functional and able to demonstrate the input-output process using the trained model.

AI Usage in Part B

The use of artificial intelligence tools is allowed during the interface development process.

However, even if students use AI tools, they must be able to explain how their interface works, how the model is integrated into the interface, and the process from user input to model output.

Presentation

Students will present their Part B work in class.

Presentation date: 18 May

During the presentation, students are expected to show:

- The code they used for model training in Part A
- Brief information about the models and hyperparameters they tested
- How they selected the best model
- The working interface they developed
- How input is provided through the interface
- How the model output is displayed in the interface

Students are not expected to show the source code of the interface. They should demonstrate the working version of the interface.

Evaluation

The Part B grade will be given based on the **in-class presentation on 18 May**.

The following points will be considered during evaluation:

- Whether the interface is functional
- Whether the interface uses the best model trained in Part A
- Whether the interface can receive input in the correct format
- Whether the model output is displayed clearly
- Whether the student can explain the model training code and experimental process
- Whether the student can explain the working logic of the interface

Students who do not attend the presentation or do not demonstrate a working interface may receive zero for Part B.